

1. Assume you are to design a CNN layer

Input: [128,128,64], now adding a 1×1 Conv layer w/ 16 kernels, padding 0, stride 1

Pick the correct answers

- a) Not possible to apply this conv layer
- b) Resulting dim: [128,128,16]
- c) 1×1 conv does not exist
- d) Better: replace 1x1 layer with 2×2 max pooling → equivalent operation, but more efficient
- e) Better: 1×1 with stride 2 → equivalent but more efficient

Solution: Only b) correct. Conv is possible. Output shape=[128,128,16]

Using stride 2 would reduce spatial resolution (from 128×128 to 64×64). 2x2 max pooling also reduces dimensions to 64x64

2. VGG Net:

- a) Predominantly 3×3 conv ☒ **Correct**
- b) Single stage approach ☒ **Incorrect. Multiple stacked 3x3 layers to replace single 5x5 layer. Receptive field is the same**
- c) Symmetric encoder-decoder structure ☒ **Incorrect**
- d) Majority of network weights located in fully connected layers ☒ **Correct**
- e) Cannot be used for image classification ☒ **Incorrect**

5. Normalization for image classification:

- a) BatchNorm used since AlexNet ☒ **Incorrect. AlexNet (2012) did not use BatchNorm — it used Local Response Normalization (LRN)**
- b) Since Xception, most image classifiers started to use LayerNorm ☒ **Incorrect. LayerNorm is mostly used in transformers or sequence models (e.g., NLP). CNN-based image classifiers like Xception, ResNet, etc., still use BatchNorm**
- c) BatchNorm used in InceptionNet to speed up training ☒ **Correct**
- d) BatchNorm part of most deep networks for image classification (e.g., ResNet) ☒ **Correct**

8. Autoencoders:

- a) Sparse AEs enforce sparsity on the representation via a regularization term. ☒
- b) Sparse AEs enforce hidden units to be inactive (i.e., activation = 0). ☒
- c) Sparse AE can be trained on a copy task, where input = target of network is identical. ☒
- d) Sparse AE force input units to be inactive (i.e., activation = 0). ☒

e) Sparse AE enforce sparsity via early stopping. ❌

f) Sparse AE cannot be trained on a denoising task, where input = noisy & target not. ❌ You can combine **sparse representations** and **denoising objectives** — they are not mutually exclusive.

15. Which of the following examples represents a common group action used in graph NNs?

a) Rotation ❌ (geometric deep learning) **could also be true**

b) Scaling ❌ (relevant for continuous data)

c) Translation ✅ Translation(permutation) is most common group action in **GNNs**. GNNs are typically **permutation invariant/equivariant**, meaning that **reordering nodes (translating indices)** doesn't change the output. **no inherent ordering** of nodes

d) Cropping ❌ (Data augmentation for images)

e) Clustering ❌ (**graph algorithm**, not a group action)

✓ a) JS & KL divergences show problems for distributions with disjoint supports — Wasserstein overcomes this.

✓ Correct

- KL and JS divergences become problematic (often undefined or infinite) when the distributions have disjoint supports.
- The Wasserstein distance remains finite and well-behaved in these cases because it considers the cost of moving mass from one distribution to another.

✗ b) A map $D : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ is a divergence if it's non-negative and symmetric.

✗ Incorrect definition of divergence

- In information theory, a divergence must satisfy:
 1. $D(p||q) \geq 0$
 2. $D(p||q) = 0$ iff $p = q$
 3. Not necessarily symmetric: KL divergence, for example, is not symmetric.
- This definition actually describes a distance, not a divergence.

It is a distance but a distance should technically also be a divergence. I guess c) should be false then

✓ c) Wasserstein-1 distance with Euclidean cost is not a divergence.

✓ Correct

- Wasserstein is a distance, not a divergence.
- It satisfies symmetry and the triangle inequality, unlike most divergences (e.g., KL).

✓ d) Wasserstein is defined as a minimization over joint distributions, but this is infeasible in practice.

✓ Correct

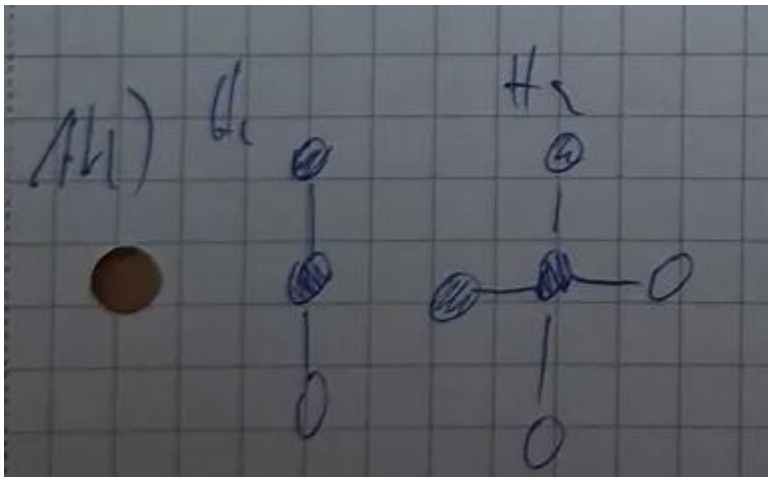
- The primal form of W_1 is a minimization over couplings (joint distributions).
- It's computationally expensive, especially in high dimensions.

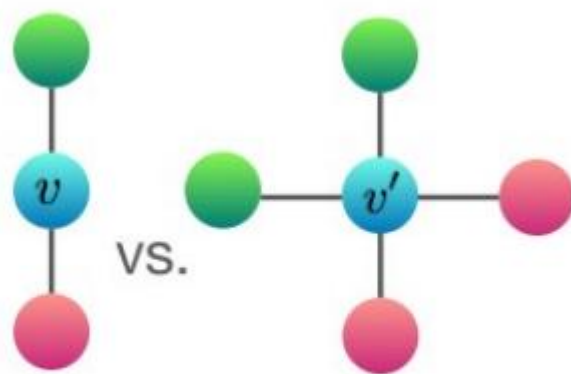
✓ e) Kantorovich–Rubinstein theorem allows formulating Wasserstein as a maximization over Lipschitz functions, making it tractable.

✓ Correct

- This is the dual formulation of W_1 , used in Wasserstein GANs.
- It makes the problem easier to optimize and more practical in deep learning.

Retry 2024





(c) Mean and Max both fail

$$\det(T) = (2)(1) - (0)(-2) = 2$$

For any matrix

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Apply this to $T = \begin{pmatrix} 2 & -2 \\ 0 & 1 \end{pmatrix}$:

- $a = 2, b = -2, c = 0, d = 1$
- $\det = 2$

$$T^{-1} = \frac{1}{2} \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & 1 \\ 0 & 1 \end{pmatrix}$$

$$p_x(x) = \frac{1}{|\det(T)|} \cdot p_u(T^{-1}x) = \frac{1}{2} \cdot p_u \left(\begin{pmatrix} \frac{1}{2} & 1 \\ 0 & 1 \end{pmatrix} x \right)$$

9/18: $u(x) = \text{ReLU} \left(\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} x \right)$ decoder $v(x) = \text{ReLU} \left(\begin{bmatrix} 1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix} x \right)$
 calculate input loss $L(x, \hat{x}) = \sum (x_i - \hat{x}_i)^2$
 for input $x = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$

Encoder:

Defined as:

- $u = \text{ReLU}(W_{\text{enc}} \cdot x)$
- Encoder weights:

$$W_{\text{enc}} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

Perform matrix-vector multiplication:

$$W_{\text{enc}} \cdot x = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 0 \cdot 1 + (-1) \cdot 2 \\ 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 2 \end{bmatrix} = \begin{bmatrix} 1 - 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Apply ReLU:

$$u = \text{ReLU}(-1, 1) = (0, 1)$$

Decoder:

Defined as:

- $\hat{x} = \text{ReLU}(W_{\text{dec}} \cdot u)$

Decoder weights:

$$W_{\text{dec}} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix}$$

Matrix multiplication:

$$W_{\text{dec}} \cdot u = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 0 + 0 \cdot 1 \\ 0 \cdot 0 + 0 \cdot 1 \\ 1 \cdot 0 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Apply ReLU (no effect, all non-negative):

$$\hat{x} = (0, 0, 0)$$

MSE Loss:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

$$x = (1, 1, 2), \quad \hat{x} = (0, 0, 0)$$

$$\text{MSE} = \frac{1}{3} [(1 - 0)^2 + (1 - 0)^2 + (2 - 0)^2] = \frac{1}{3} [1 + 1 + 4] = \frac{6}{3} = 2.0$$

Should not have used MSE loss. So division by 1/n was false. Solution should be 6